

SCRAPING LOGIC

1. Selenium Web Automation

The project uses Selenium WebDriver to automate browser actions.

Main browser actions:

- Open website
- Search products
- Scroll pages
- Extract HTML elements
- Handle dynamic content
- Close browser safely

Browser used: Google Chrome

Driver management: webdriver-manager automatically downloads and manages ChromeDriver.

2. Headless Browsing

The scraper runs in headless mode (`--headless`)

- Prevent browser window from opening
- Improve performance
- Reduce memory usage

3. Anti-Bot Detection Techniques:-

E-commerce websites detect automated scraping. To reduce bot detection, the project uses:

- a) Custom User-Agent:- Pretends to be a real Chrome browser.
- b) Hide WebDriver Property:- Removes Selenium automation traces from JavaScript.
- c) Random Delays:- Adds random waiting times between actions.

Example: Waiting between products, waiting after scrolling, waiting between websites

- d) Controlled Scrolling:- Loads products gradually like a real user.

4. Amazon Scraping Logic

- a) Generate Amazon search URL.

Example: <https://www.amazon.in/s?k=gaming+laptop>

- b) Open URL using Selenium.
- c) Wait until product containers load.

Selector used: [data-component-type="s-search-result"]

- d) Loop through each product container.

- e) Extract:

- Product name

- Price

- Rating
- f) Clean extracted data.
- g) Validate: Price range, product type, missing values
- h) Save valid products into DataFrame.

4.1 Amazon Product Name Extraction

The scraper tries multiple CSS selectors because Amazon frequently changes HTML structure.

Selectors used:

- h2 a span
- h2 a
- .a-size-medium
- .a-size-base-plus
- .a-text-normal

Logic: Extract text, remove whitespace, ignore invalid names, return first valid match

4.2 Amazon Price Extraction

Selectors used:

- span.a-price-whole

- .a-price .a-offscreen
- .a-price .a-price-whole

Logic:

- Extract raw text
- Remove: ₹, commas, spaces
- Convert to numeric value

Example: ₹64,990 → 64990

4.3 Amazon Rating Extraction

Selector: span.a-icon-alt

Logic:

- Extract rating text
- Use regex to get numeric value

Example: "4.3 out of 5 stars" → 4.3

5. Flipkart Scraping Logic

a) Generate Flipkart search URL.

Example: <https://www.flipkart.com/search?q=gaming+laptop>

b) Open URL.

c) Handle login popup automatically.

- d) Scroll dynamically to load products.
- e) Extract product containers.
- f) Extract: Product name, Price, Rating
- g) Validate laptop products.
- h) Remove duplicate products.
- i) Store data in DataFrame.

5.1. Flipkart Login Popup Handling

Flipkart often shows a login popup.

Logic:

- Wait for popup close button
- Click close button automatically
- Continue scraping

Selector: `button._2KpZ6l`

5.2 Flipkart Dynamic Scrolling

Flipkart loads products dynamically while scrolling.

Logic:

- Scroll to bottom
- Wait for new products

- Check page height
- Stop when no new content loads

Benefits:

- Loads more products
- Mimics human browsing

5.3 Flipkart Product Name Extraction

Selectors used:

- a._1fQZEK
- .s1Q9rs
- ._4rR01T
- .IRpwTa
- a[title]

Fallback logic:

- Read full product text
- Search for laptop keywords

5.4 Flipkart Price Extraction

Primary logic: Search entire product text using regex

Regex: ₹ followed by digits

Fallback selectors:

- div._30jeq3
- div._25b18c

Additional logic: Correct extra-zero pricing errors

6. Safe Element Extraction

The scraper never crashes if elements are missing.

Functions:

- safe_find_element()
- safe_find_elements()

Purpose:

- Prevent exceptions
- Return None or empty list safely

7. Retry Mechanism

The project retries scraping automatically if failure occurs.

Retry count: 3 attempts

Delay between retries: 5 seconds

Benefits:

- Handles temporary website issues

- Handles slow loading pages
- Improves scraping reliability

8. Data Cleaning Logic

Cleaning steps:

- Remove extra spaces
- Remove currency symbols
- Remove commas
- Convert prices to numeric values
- Normalize product names

9. Product Validation Logic

The scraper validates each product before saving.

Validation rules:

- Product name must exist
- Price must exist
- Price must be within laptop range
- Product must not be an accessory

10. Accessory Filtering

The scraper removes accessories accidentally returned in search results.

Filtered keywords: keyboard, mouse, adapter, cable, bag, monitor, charger

Purpose: - Keep only laptop products

11. Duplicate Removal

Flipkart often repeats products.

- Track seen prices
- Remove repeated entries
- Drop duplicate rows

12. Product Matching Logic

The project compares products across platforms.

Techniques used:

1. Model Number Extraction
2. Fuzzy String Matching

13. Model Number Extraction

Regex patterns identify laptop model numbers.

Examples: G15, XPS-13

Purpose: Improve matching accuracy

14. Fuzzy Matching

Uses FuzzyWuzzy library.

Function: token_set_ratio()

Purpose: Compare product names even if wording differs

15. Match Confidence Score

Each matched product gets a confidence score.

Range: 0% to 100%

Higher score: Higher probability of same product

16. Error Handling

The scraper uses try-except blocks extensively.

- Prevents crashes
- Skips broken products
- Continues scraping safely